a)

Primary Actor: User

Secondary Actor: File System

Main Use Case: Solve Puzzle Files

b)

The user runs the program. For each puzzle file, the system reads the 9×9 grid, checks that the clues don't already conflict, and finds every valid solution. It then prints the file name, the original puzzle, and each solution. If none exists, it prints "No solution found".

c)

1. The User starts the program, passing filenames.

2. For each file, the system loads the 9×9 grid from the File system.

3. The system validates that the given clues contain no row, column or block conflicts.

4. The system picks the empty cell with the fewest candidates.

5. The system tries each candidate digit and recurses. If a branch has no candidates, it backtracks.

6. When all 81 cells are filled, the system saves that solution and keeps searching for others.

7. When the search is exhausted, the system prints the file name, the puzzle, and every solution found.

8. The system moves on to the next file. After the last file, the program ends.